Paws Rescue Donations

Brief: What It Is, How It Works, and Where It Breaks

Explainer for `paws-rescue-donations`

The short version

In one paragraph

A Django web application that replaces a pet rescue's donation spreadsheet with a database behind a login. Staff record who gave how much for which animal, filter and sort those records, watch running totals, and export the result as CSV. Built for a faculty-supervised Khidmat community-service project at Habib University. It is small, conventional and competently made. It also carries two real bugs, both verified by running the code, both sitting in the only two features nobody wrote a test for.

This document assumes no prior knowledge; a glossary at the end defines every technical term it uses. Its companion, the Deep Dive, walks the same project function by function.

1 Why it exists

Small rescues track donations in a spreadsheet. That works until two or three people are entering records, at which point three failures arrive at once: no access control, no consistent fields, and no reliable way to ask the data a question such as "how much has Bella received this month".

This project answers that with the smallest thing that could work: one database, one login, one dashboard. There is no public side at all. No donor-facing page, no sign-up form. Every route is behind staff authentication.

The complete feature set

Email and password login; create, edit and delete donations; filter by seven criteria at once; sort and paginate; a dashboard with running totals and a top-5-pets panel; CSV export of the filtered view; add and remove staff logins from the web UI.

2 How it is built

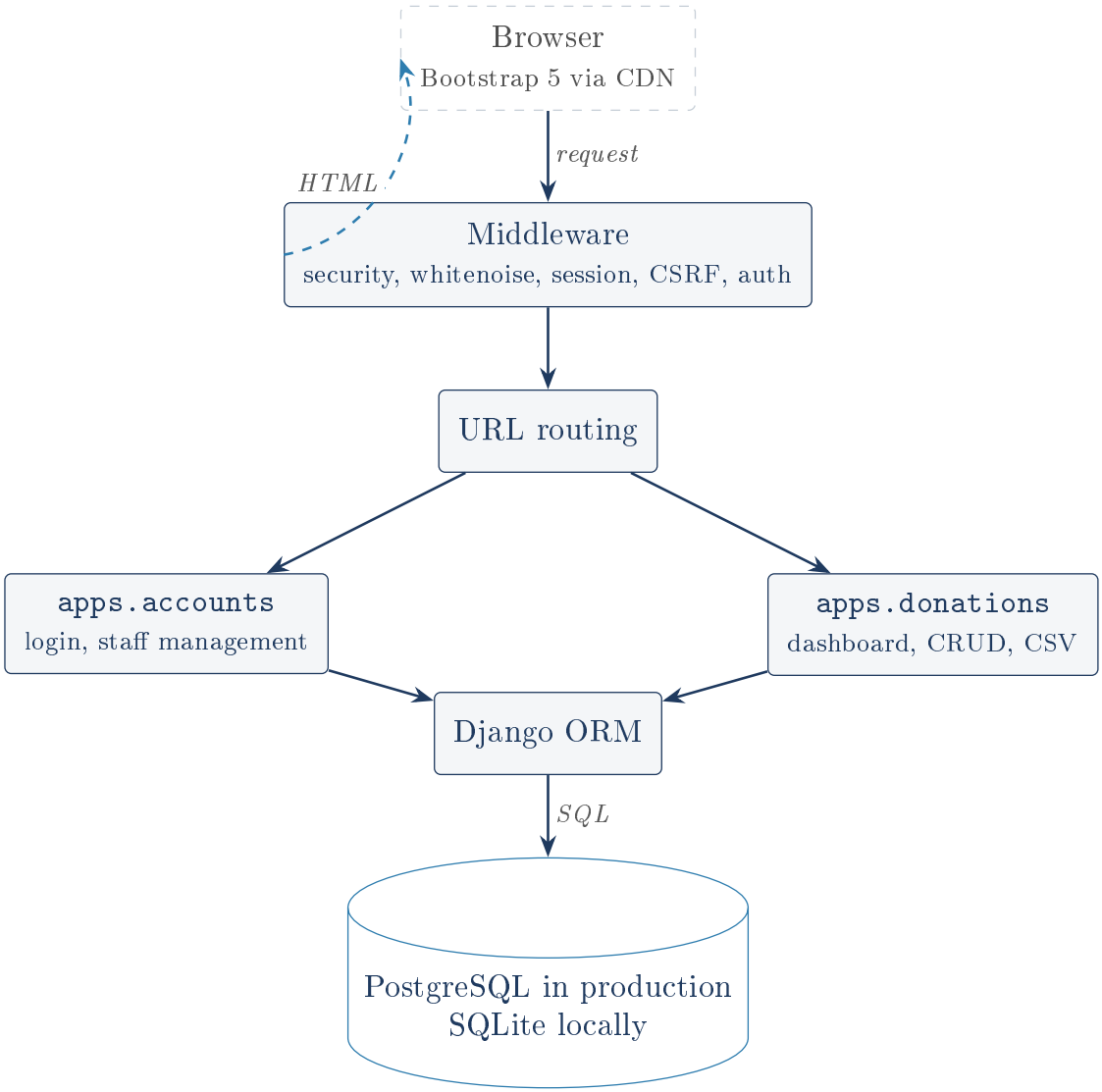

Figure 1: Everything is server-rendered. The browser receives finished HTML and runs no JavaScript the project wrote itself.

Two Django apps carry all the behaviour. `accounts` decides who may log in; `donations` is the subject matter. A third folder, `paws_rescue`, holds configuration rather than features. Six dependencies, all pinned: Django, psycopg2, python-dotenv, gunicorn, whitenoise, dj-database-url.

2.1 The whole data model

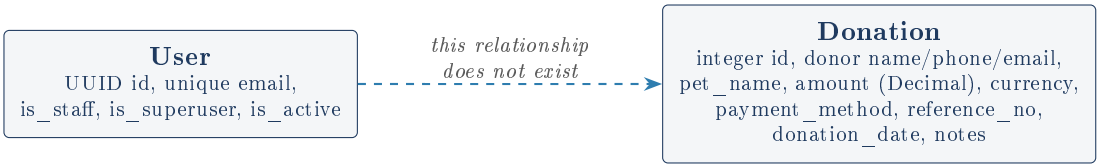

Figure 2: Two tables, and the missing link between them: nothing records which staff member entered a donation.

2.2 Three decisions worth understanding

A custom user model, defined on day one. Django's built-in user has a `username` field; this app wants email-only login. So it defines its own user with `USERNAME_FIELD = 'email'`, a UUID primary key, and a hand-written manager to replace the default one (whose `create_user(username, ...)` signature no longer fits). This matters more than it looks: `AUTH_USER_MODEL` is close to unchangeable after the first migration, so it is the highest-leverage decision in a new Django project, and getting it right at the start is a genuine credit to the build.

Money is a `Decimal`, never a float. Floats store values in binary, where 0.1 has no exact representation and `0.1 + 0.2` yields `0.30000000000000004`. Summing thousands of donations that way accumulates real error. `DecimalField` stores the digits exactly.

Filtering rides on QuerySet laziness. `dashboard_view` stacks up to seven filters, each applied only if the user supplied that field. None of them touch the database. A Django QuerySet records conditions and runs nothing until something reads it, so all seven collapse into a single SQL query with one `WHERE` clause.

2.3 The dashboard, which is where the work happens

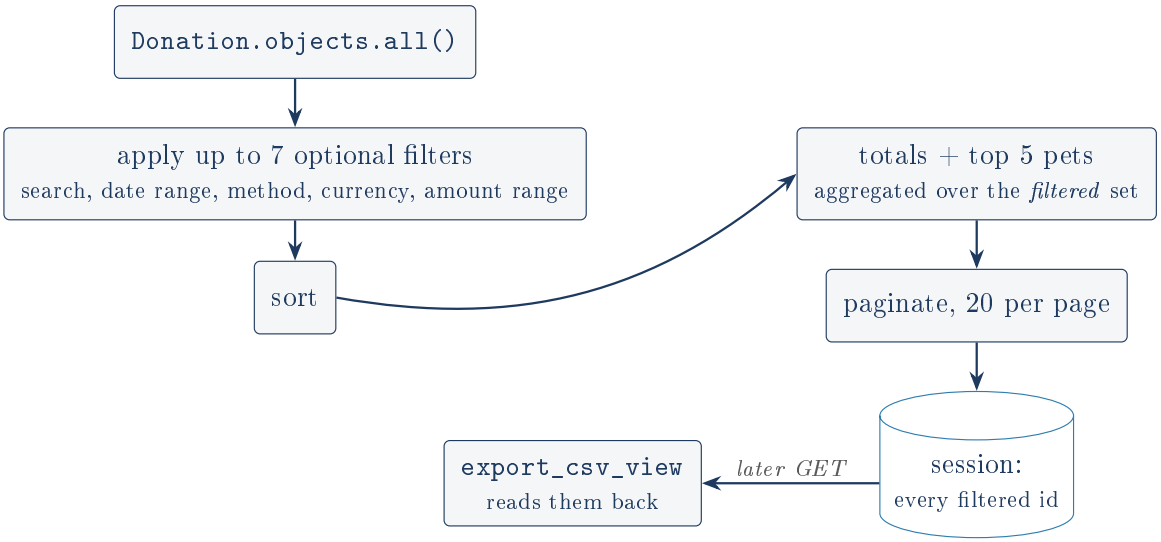

Figure 3: One view does filtering, aggregation, pagination and the session handoff that feeds CSV export. That last arrow is where both of the project's serious bugs live.

The CSV export link is a plain GET carrying no filter state, so the export view cannot know what the dashboard was showing. The workaround: the dashboard writes every matching donation id into the session, and the export view reads them back. The README already calls this "the weakest part of the design". It is right, and running it shows the weakness is more concrete than the README realised.

3 Verification: what was actually run

Nothing in this document is taken on trust from the README. A clean virtual environment was built, the pinned dependencies installed, and the code executed.

```
$ python manage.py test
Ran 11 tests in 12.828s

OK
```

Listing 1: The test suite, run on this machine

Eleven tests, all passing, exactly as the README claims. They cover authentication redirects, admin creation, the self-deletion guard, donation CRUD, search, and the CSV response headers. The dependency pins install cleanly.

That is a real result and it is also the setup for the section below: the suite is green and the two worst bugs are untouched by it.

4 Where it breaks

A filter matching nothing exports the entire database

`export_csv_view` reads the session ids and branches on `if filtered_ids:`. An empty list is falsy, so it cannot distinguish "the user never loaded the dashboard, export everything" from "the user's filter matched zero rows, export nothing". Driven through the real test client with two donations present:

```
GET /dashboard/?search=NoSuchDonorZZZ   -> page shows "0 donation(s)", session ids == []
GET /donations/export/                  -> CSV contains 2 data rows (every donor)
```

The user filters to nothing, sees "No donations found", clicks Export CSV, and silently downloads every donor name, phone number and email in the system. It is triggered by an ordinary action, it fails in the unsafe direction, and it is invisible because the file downloads without complaint. This is the most serious defect in the project and the fix is one line.

Mixed currencies are added together and labelled PKR

`Sum('amount')` ignores the `currency` column next to it, and the dashboard template appends the literal text "PKR" to the result. With one 100 PKR donation and one 100 USD donation present:

```
total_all_time: 200        rendered page contains "200.00 PKR"? True
```

The dashboard reports 200.00 PKR. The rescue received 100 PKR and 100 USD. A confidently presented wrong number is worse than an error.

The README's own account of the currency issue is backwards

The README says a free-text currency field means typos such as "pkr" or "Rs" "would silently split the totals". Running it shows the opposite. Nothing splits; everything merges, because the totals never group by currency in the first place. And `currency__iexact` means the filter already folds "pkr" into "PKR" correctly, so the case-typo case is the one that works. This is worth flagging plainly: an interviewer will read the README as the author's own assessment, and defending a self-assessment the code contradicts is harder than having none. The real fix is not just a choice list; even with a clean enum, summing across currencies is still meaningless. The dashboard must group by currency or convert at a recorded rate.

The development database is committed, hash and all

`db.sqlite3` is tracked in git and `.gitignore` does not exclude it. It was added deliberately (commit `a49871e`). Reading it directly: one user row for `admin@example.com`, superuser, with a real PBKDF2 password hash; two expired session rows; zero donations. The exposure is genuinely limited (placeholder email, no donor data, expired sessions, 600,000-iteration hash) so this is not an emergency. It is still a committed credential hash and a bad habit, and the next person who commits that file after entering real donor data publishes real donor data.

Every web-created admin is an unvalidated superuser

`settings.py` configures the four standard password validators, but they only run where something calls them, and `AdminCreationForm` never does: it subclasses `ModelForm`, not Django's `UserCreationForm`. Verified by running the form: a one-character password `'1'` validates cleanly, no errors. Meanwhile the form hard-codes `is_staff = True` and `is_superuser = True` on every account it creates, so there is exactly one privilege level. The two compound into the weakest possible password on the most privileged possible account. The validators do run for `manage.py createsuperuser`, which is why the gap is easy to miss: the settings look correct.

No audit trail

`Donation` has no foreign key to `User`. Nothing records who entered or deleted a record, and deletes are hard deletes with no history. For a system whose entire purpose is tracking other people's money, this is the largest structural gap, and it is the first thing a reviewer will ask about. The README raises it too.

Smaller, and real

`SECRET_KEY` falls back to a hard-coded value published in the repo, so a deployment that forgets the variable runs happily on a known key. The session export grows without bound and is rewritten to the database on every request (`SESSION_SAVE_EVERY_REQUEST = True`). `donation_date` and `pet_name` drive the ordering and the grouping and neither is indexed. `logout_view` still accepts GET. `UserCreationForm` is imported and unused, `is_admin` is defined twice, and `django.contrib.admin` is routed at `/admin/` with nothing registered in it.

4.1 Why a green suite missed all of this

The tests assert on page text, and on the easy branch

Every assertion is a substring match against the whole HTML response, chrome included. The project already has a scar from this: `assertNotContains(response, 'Max')` once failed on a page with no donation for the pet Max, because the filter form's "Max Amount" label contained the substring. It was resolved by renaming the label to "Amount To". That fix is backwards, and worth admitting: the user interface was changed to suit the test's matching strategy, and the test is still fragile.

Beyond fragility, three tests assert less than their names promise. `test_filter_by_payment_method` checks that the cash donor appears but never that the online donor is absent, so a filter that ignored its parameter entirely would pass. `test_amount_validation` checks only that no row was created, so a view that crashed with a 500 would pass. And `test_csv_export_...` exercises the export with no filter applied, which is the branch that works.

That last one is the whole lesson. Eleven green tests bought real confidence about authentication and CRUD, and no confidence at all about the two features most likely to put a wrong number, or somebody's phone number, in front of a user.

5 What is genuinely good

Said plainly, because it is not padding. The custom user model was done on day one, which is the one Django decision that is nearly impossible to walk back. Money is a `Decimal`. The amount filters use `if amount_min is not None` rather than a truthiness test, a deliberate guard against

falsy zero showing the author was thinking about edge cases. The self-deletion guard is enforced in the view, not merely hidden in the template. The top-pets panel once aggregated over the unfiltered table and contradicted the total beside it; noticing and fixing that is exactly the class of bug most people ship.

And the `TESTING = 'test' in sys.argv` flag is a real debugging win. Working out why every test suddenly 301-redirects to https requires knowing that `manage.py test` does *not* set `DEBUG=True`, so a block guarded by `if not DEBUG` applies during tests and `SECURE_SSL_REDIRECT` silently breaks the entire suite. That is a non-obvious chain, and the README reconstructs it correctly.

The README volunteers several of its own weaknesses, including the missing foreign key, the session export and the missing indexes. That is rarer than it should be and worth defending as deliberate practice, which is exactly why the one place it gets a detail backwards (the currency claim) is worth correcting before someone else finds it.

6 If it continued

1. Fix the empty-filter export. Live data-exposure bug, one-line change, and add the test that catches it.
2. Group totals by currency, or constrain the field and then group. Stop printing a number that is not true.
3. Untrack `db.sqlite3`.
4. Validate admin passwords; stop making every admin a superuser.
5. Add `entered_by` to `Donation`; stop hard-deleting.
6. Re-derive the export from the query string and delete the session handoff, which retires two findings at once.
7. Make `SECRET_KEY` mandatory in production; index `donation_date` and `pet_name`; split settings into base, dev and prod.

The one-sentence summary

A small, conventional, competently built Django CRUD app that gets several non-obvious Django decisions right and carries two real bugs in the two places nobody tested: an empty filter exports everything, and mixed currencies are added together and labelled PKR.

Glossary

Django
A Python web framework: a pre-built skeleton handling the plumbing (listening for requests, matching URLs, talking to the database, rendering HTML) so you write only what is specific to your problem.

App
A Django folder grouping the models, views and templates for one area of responsibility. Not a separate program. One project contains many.

Model
A Python class describing one kind of record, which Django maps onto a database table.

View
A Python function that receives a request, does the work, and returns a response.

Template
An HTML file with placeholders filled in with data at render time. Django escapes values by default, which is what makes displaying donor-entered names safe.

ORM
Object-Relational Mapper. Lets you write `Donation.objects.filter(pet_name='Max')` instead of SQL, and returns Python objects rather than rows.

QuerySet
What the ORM returns. Lazy: it records conditions and does not query the database until something reads the results.

Migration
A recorded, versioned instruction for changing the database structure, generated from the models and applied in order.

Middleware
Components every request passes through on the way in and every response on the way out, handling concerns no single view should repeat.

Session
Server-side storage tied to one browser by a cookie. The browser holds an opaque key; the data lives in a database table.

CSRF
Cross-Site Request Forgery: an attack where another site quietly submits a request to one you are logged into, riding your cookie. Django defends with a secret token in every form.

Foreign key
A column holding the identifier of a row in another table, expressing "this donation was entered by that user". The relationship this project lacks.

Decimal vs float
Decimals store digits exactly; floats store binary approximations and drift. Money uses Decimal.

UUID
A random 128-bit identifier used instead of a sequential integer, so ids cannot be enumerated or guessed.

Password hashing
Storing a one-way scrambled form of a password. Django uses PBKDF2-SHA256 at 600,000 iterations; the deliberate slowness makes guessing expensive.

Whitenoise
A library letting the Python app serve its own CSS and images competently, so no separate web server is needed.

collectstatic
A build step gathering static files into one directory; the manifest variant also hashes filenames for caching, and raises an error if the build step never ran.

Gunicorn
The production server that runs the Python application, named in the `Procfile`.

CRUD
Create, Read, Update, Delete.

Every claim here was checked against the source, and every claim marked verified was established by executing the code: the test suite, the form validation probes, the export behaviour, the mixed-currency totals, and the contents of the committed database.